Syntax Skeleton Sheet: Express + Mongoose + HTML

Real syntax shape with the key parts blanked out. Fill each ____ from memory.

1. Index.js

Purpose	Syntax skeleton (fill the blanks)
Import express	`const express = require("____");`
Import mongoose, path	`const ____ = require("____"); // x2`
Import a model	`const User = require("./____/____");`
Create app	`const app = ____();`
Parse form data	`app.use(express.____({extended:____}));`
Parse JSON	`app.use(express.____());`

Connect to DB	`mongoose.____("mongodb://127.0.0.1:____/____").then(() => {`
	`  ...routes + listen go here...`
	`}).____(console.error);`

Redirect route	`app.get("/", (req,res) => res.____("/____"));`
Open an HTML page	`app.get("/login", (req,res) => res.____(path.join(____, "login.html")));`

Register user	`app.post("/registration", ____ (req,res) => {`
	`  ____ User.____(req.body);`
	`  res.redirect("/____");`
	`});`

Login user	`app.post("/login", async (req,res) => {`
	`  let u = await User.____(req.body);`
	`  if (!u) return res.____("Invalid Login");`
	`  res.redirect("/____");`
	`});`

Fetch all records	`app.get("/items", async (req,res) =>`
	`  res.____(await Model.____())`
	`);`

Save a record	`app.post("/add", async (req,res) => {`
	`  await Model.____(req.body);`
	`  res.send("____");`
	`});`

Start server	`app.____(____, () => console.log("http://localhost:____"));`

2. Model files

Purpose	Syntax skeleton (fill the blanks)

User schema	`module.exports = mongoose.____("User", new mongoose.____({`
	`  ____: String,`
	`  ____: String`
	`}));`

Domain schema	`module.exports = mongoose.model("____", new mongoose.Schema({`
	`  ____: ____, // one line per field in the question`
	`}));`

Hint: User has 2 fields. The domain model (e.g. Sports) has 4 string fields, the last being optional details.

3. Script in index.html

Purpose	Syntax skeleton (fill the blanks)

Load and show rows	`async function ____() {`
	`  let d = await (await ____("/items")).____();`
	`  data.____ = d.____(x =>`
	`` `<tr><td>${x.____}</td>...</tr>`).join("");``
	`}`

Submit handler	`form.____ = async e => {`
	`  e.____();`
	`  await fetch("/____", {`
	`  method:"____",`
	`  headers:{"Content-Type":"____"},`
	`  body:JSON.____(Object.____(new ____(form)))`
	`  });`
	`  form.____();`
	`  ____();`
	`};`

Run on page open	`____(); // last line of the script`

Hint: the HTML needs a <tbody id="____"> and a <form id="____"> that the script refers to by id.

4. Login / Registration pages

Purpose	Syntax skeleton (fill the blanks)
Head	`<title>____</title>` `<link rel="____" href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/____.min.css">`
Body	`<body class="____ mt-____">`
Form	`<form action="/____" method="____">`
Username input	`<input class="form-control mb-3" name="____" placeholder="Username" ____>`
Password input	`<input class="form-control mb-3" type="____" name="____" placeholder="Password" required>`
Button	`<button class="btn btn-____">Login</button>  // success for registration`
Switch page link	`<p class="mt-3">New user? <a href="/____">Register here</a></p>`

5. Self-check

- Do the input name values match the schema field names exactly?
- Is every route that touches the database marked async, with await before the call?
- Does the require path match the model file name?
- Is app.listen placed inside the .then() of mongoose connect?